Online Appendix

Coding open-ended responses with large language models: Evidence from three economic experiments

Daniel Parra Sophia Aristizabal

Appendix A Validation metrics: definitions

This section provides the formal definitions of the performance metrics used in the main text. The most straightforward metric is accuracy, which measures the proportion of correct classifications regardless of the response type (i.e., whether the response truly belongs to the category, TP, or does not belong to it, TN):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

This metric is most informative when the number of positive and negative cases is relatively balanced, as it may otherwise obscure poor performance in minority classes. For example, if 90% of the responses belong to a given category and the LLM classifies all responses as belonging to that category, the accuracy will still be high. However, this result masks the model's inability to correctly identify responses that do not belong to the category.

Precision is the proportion of positive classifications made by the model that are actually correct within a given category:

$$\text{Precision} = \frac{TP}{TP + FP}$$

This metric is particularly useful when it is important to avoid incorrectly assigning responses to a category. However, precision does not indicate whether some responses that truly belong to the category were not classified as such. For example, if two responses are correctly assigned to a label while 98 other relevant responses are missed, precision will still be 100%. This limitation can be addressed by considering recall, which measures the

proportion of actual positive cases that are correctly identified by the LLM:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Finally, when both precision and recall must be considered jointly, the F1-score represents the harmonic mean of the two:

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

An F1-score closer to one indicates that both recall and precision are high.

Cohen's kappa measures agreement between two raters, with values ranging from 1 (perfect agreement) to 0 (agreement equivalent to chance), while negative values indicate systematic disagreement between coders. Krippendorff's alpha is appropriate when more than two coders are involved, and its values are interpreted similarly.

Appendix B Prompt templates

This is the general structure that was used to create the prompts of the three empirical applications:

```
// Role
You are <role description>

// Context
<Game structure>
<Treatment Conditions>
<Communication type>
<Underlying experimental logic>

// Classification Task
<Language note>
<Participants responses' structure>
<Categories definition>

// Format
Please give your answer strictly as a python dictionary, with no
additional text or characters.  Use the same tags you chose for each
category.
<JSON structure>

// Constraints
<Constraints specification>
```

These are the prompts used for each case study:

B.1 Prompt for (Brandts and Cooper, 2025)

```
// Role
You are an expert research assistant specializing in qualitative
analysis of behavioral economics experiments, with extensive experience
interpreting and coding free-form participant responses.

// Context
This experiment examines coordination between a manager and two agents
under conditions of conflicting preferences and asymmetric information.
Participants (567 undergraduate students in 189 groups of three) were
assigned fixed roles (one manager M, two agents S1 and S2) and played
```

18 rounds where the game state varied randomly.

GAME STRUCTURE:
- All parties benefit when agents coordinate on a common action
- Agents have opposing preferences over which coordinated outcome to choose
- Agents observe the game state; the manager does not
- The efficient outcome (maximizing total surplus) varies by state
- Manager's objective is to maximize combined agent payoffs

TREATMENT CONDITIONS:
Groups were assigned to different conditions varying:

Communication type:
- No communication (baseline)
- Structured communication (predefined message options)
- Free-form chat (unrestricted text messaging)

Control mechanism:
- Delegation (agents choose actions)
- Delegation with managerial advice
- Managerial control (manager selects actions)

After each round, participants received feedback on the game state and actions. In some treatments, managers could observe discrepancies between agents' reported information and the actual state.

// Classification Task
Language note: The input messages are primarily in Spanish. Interpret the content in Spanish from Spain and output only the required labels in the specified format.

The task you have is to classify each group of messages sent in one or more of the following categories. The group of messages represent what the participants communicated sequentially during each period. Additionally, it states who sent the message: 0 if it is Central Manager, 1 if it is agent 1, and 2 if it is agent 2.

These are the categories stated:

1. Did they make a suggestion about what row/column should be chosen? (any_suggestion) If it was the case:
a. Did they suggest a safe outcome? (suggest_safe)
b. Did they suggest an efficient outcome? (suggest_efficient)

2. Did they agree to the proposal about what row/column should be chosen? (agree_proposal)
3. Did they discuss about what row/column should be chosen? If it was the case:
a. Did they discuss need to coordinate? (discuss_coordinate) IMPORTANT BOUNDARY: This requires more than making a suggestion that involves coordination. The message needs to indicate that the two players should be choosing the same thing (e.g. "We'll do better if we make the same choices" is coded. "Let's choose row 4 and column 4" is not coded.)
b. Did they discuss fairness? (discuss_fairness) This category includes any message that discusses the distribution of pay over the three players. Use 0.5 if fairness is raised only briefly or incidentally.
c. Did they discuss efficiency? (discuss_efficient) This includes discussion of maximizing total pay as well as explaining how and why rotation between players works. Use 0.5 if efficiency is mentioned in passing.
4. Did they send questions about rules of the experiment? (discuss_rules) Use 0.5 if the question is ambiguous (could be asking about rules or about strategy).
5. Did they give any explanation (explanation)? This included explanations about the rules of the experiment or game, as well as explanations of a suggested way of playing the game. Use 0.5 if the explanation is partial or unclear.
6. Did they send questions about how to play? (discuss_howtoplay) This was for conceptual questions rather than the frequent generic request that somebody suggest a row and column.
7. Did the Manager (coded as 0) ask what game is being Played?

(ask_game) Use 0.5 if the question is implicit or ambiguous.
8. Did Agents Report What Game is Being Played? (receive_report) If that is the case:
a. Did they truthfully Reveal Game? (truthful)
b. Did they lie About Game? (falsehood)
c. Conflict (contradict): The agents contradict each other: one says one game, the other says a different game. (Note: previously mislabeled 'contradict' was the fact-checking category; see constraint file for final mapping.)
9. None of the agents report. This is true if in the last question "Did Agents Report What Game is Being Played?", the answer was false. (neither_report)

// Format
Please provide your answer strictly as a Python dictionary with no additional text or characters.

Use these exact keys with no extra spaces:
- 'any_suggestion'
- 'suggest_safe'
- 'suggest_efficient'
- 'agree_proposal'
- 'discuss_coordinate'
- 'discuss_fairness'
- 'discuss_efficient'
- 'discuss_rules'
- 'explanation'
- 'discuss_howtoplay'
- 'ask_game'
- 'receive_report'
- 'truthful'
- 'falsehood'
- 'contradict'
- 'neither_report'

All values must be:
- 1 if the category is clearly observed

- 0 if the category is not observed

Be sure that each tag is in simple quotes.
Example format:
{
'any_suggestion': 1,
'suggest_safe': 0,
'suggest_efficient': 1,
'agree_proposal': 0,
'discuss_coordinate': 1,
'discuss_fairness': 1,
'discuss_efficient': 0,
'discuss_rules': 1,
'explanation': 1,
'discuss_howtoplay': 1,
'ask_game': 0,
'receive_report': 0,
'truthful': 0,
'falsehood': 0,
'contradict': 0,
'neither_report': 1
}

// Constraints
LOGICAL CONSTRAINTS:

1. Suggestion hierarchy:
- If 'any_suggestion': 0, then both 'suggest_safe': 0 and
'suggest_efficient': 0
- If 'any_suggestion': 1, then 'suggest_safe' and 'suggest_efficient'
can be 0 or 1 independently

2. Reporting logic:
- 'receive_report' and 'neither_report' are mutually exclusive:
• If 'receive_report': 0, then 'neither_report': 1
• If 'receive_report': 1, then 'neither_report': 0

```
3. Report content (only applies when 'receive_report': 1):
- Exactly one of the following must equal 1, the others must be 0:
• 'truthful'
• 'falsehood'
• 'contradict'
- If 'receive_report': 0, then all three must be 0
```

B.2 Prompt for (Ederer and Schneider, 2022)

```
// Role
You are an expert research assistant specializing in qualitative
analysis of behavioral economics experiments, with extensive experience
interpreting and coding free-form participant responses.

// Context
This experiment examines how communication influences trust and
cooperation in a sequential decision-making game, with particular focus
on how time delays affect these dynamics.

EXPERIMENTAL STRUCTURE:
Participants play a trust game in pairs: a Trustor (sender) and
a Trustee (responder). The game occurs in a single lab session,
supplemented by two online questionnaires (Q1 and Q2) administered
at different times.

GAME MECHANICS:
The Trustor chooses between:
- OUT: Both receive $15
- IN: Decision passes to the Trustee

If the Trustor chooses IN, the Trustee decides between:
- DON'T ROLL: Trustee receives $42, Trustor receives $0
- ROLL: Trustee receives $30, Trustor's payoff depends on a dice roll
(5/6 probability of $36, 1/6 probability of $0)
```

TREATMENT CONDITIONS:
Treatments vary across two dimensions:

Communication:
- No message condition
- Message condition (Trustee can send a message to the Trustor before the Trustor's decision)

Decision timing:
- Immediate: Trustee decides during the lab session
- 24-hour delay: Trustee decides within 24 hours after the session
- 21-day delay: Trustee decides 21 days after the session

ANALYSIS FOCUS:
Messages sent by Trustees in the communication conditions are analyzed for content and framing.

// Classification Task
Language note: The input messages are primarily in English. Interpret the content in English and output only the required labels in the specified format.

Your task is to classify each message into one or more of the following categories. Messages may belong to multiple categories.

CATEGORIES:

1. is_promise: Message in which the Trustee explicitly or implicitly commits to a specific future action. This includes direct statements (e.g., "I will choose ROLL") as well as implicit commitments that suggest intention to take a particular action.

2. appeal_efficiency: Message emphasizes practical benefits or optimal resource allocation. This includes arguments that cooperation leads to better outcomes, maximizes total payoffs, or makes better use of available resources for the parties involved.

3. appeal_fairness: Message invokes principles of fairness, equity, or moral rightness. This includes arguments that a particular choice is just, ethically appropriate, or represents equal treatment between the parties.

4. uninformative: Message is nonsensical (e.g., random text, gibberish), irrelevant to the decision context, or indicates the sender misunderstood the task.

// Format
Please provide your answer strictly as a Python dictionary with no additional text or characters.

Use these exact keys:
- 'is_promise'
- 'is_efficiency'
- 'is_ethics'
- 'uninformative'

All values must be:
- 1 if the category is observed in the message
- 0 if the category is not observed

Be sure that each tag is in simple quotes.
Example format:

'is_promise': 1,
'is_efficiency': 1,
'is_ethics': 0,
'uninformative': 0

// Constraints
LOGICAL CONSTRAINTS:

1. Uninformative messages:
- If 'uninformative': 1, then all other categories must be 0

```
- A nonsensical or irrelevant message cannot simultaneously contain
promises or appeals

2.  Multiple categories:
- Messages can contain combinations of 'is_promise', 'is_efficiency',
and 'is_ethics'
- Example:  A promise that also appeals to efficiency would have both
set to 1
```

B.3 Prompt for (Ling et al., 2025)

```
// Role
You are an expert research assistant specializing in qualitative
analysis of behavioral economics experiments, with extensive experience
interpreting and coding free-form participant responses.

// Context
The experiment explored whether undergraduate students underreport AI
usage in academic settings.  Participants (78% STEM majors) reported
their own AI reliance and frequency of use, then estimated their peers'
AI usage.

A follow-up survey asked a separate sample of undergraduate students
to explain potential gaps between self-reported and peer-attributed AI
usage through open-ended responses.  Responses may include references
to social desirability, privacy concerns, genuine lack of knowledge
about peers, or alternative explanations.

// Classification Task
Language note:  The input responses are primarily in English.
Interpret the content in English and output only the required labels
in the specified format.

Your task is to classify each qualitative response into one or more of
the following categories.  Responses may belong to multiple categories.
```

CATEGORIES:

1. uninformative: Response is nonsensical (e.g., random text, gibberish) or indicates misunderstanding of the question (e.g., explaining the gap in reverse direction, irrelevant content).

2. SDB: The response indicates that individuals may misreport or conceal AI usage in order to avoid negative social evaluation, judgment, or reputational harm.

3. overest_others: Response indicates students overestimate their peers' AI usage.

4. underest_own: Response indicates students underestimate their own AI usage.

5. academic_integrity: Response mentions academic integrity concerns, such as viewing AI usage as cheating, dishonest, or academically problematic.

6. info_asymmetry: Response indicates students lack knowledge about peers' actual AI usage patterns.

7. AI_discussion_priming: Response that attributes the gap between self-reported and peer-reported AI use to the visibility and frequency of AI as a topic of conversation, debate, or media coverage. It captures instances where respondents believe that "talking about AI" is being mistaken for "using AI".

8. privacy_concerns: Response mentions privacy concerns as a reason for underreporting personal AI usage.

9. self_esteem: Response indicates students underreport AI usage to protect self-image or self-worth (similar to SDB but focused on internal self-perception rather than external judgment).

10. self_report_bias: Response mentions self-favoring or self-enhancement bias in reporting behavior.

11. network_effect: Response explains the reporting gap by suggesting that a student's local social circle or friend group heavily influences their perception of how much the general population uses AI. It describes a scenario where high usage within a specific "network" leads a student to incorrectly assume that those same high usage rates apply to the entire student body.

12. truthful: Response indicates the gap accurately reflects real differences in AI usage: respondents genuinely use AI less than their peers.

// Format
Please give your answer strictly as a python dictionary, with no additional text or characters. Use exactly the following tags in the dictionary:
4. 'uninformative' for uninformative
5. 'SDB' for SDB
6. 'overest_others' for overest. others
7. 'underest_own' for underest. own
8. 'academic_integrity' for academic integrity
9. 'info_asymmetry' for info asymmetry
10. 'AI_discussion_priming ' for AI discussion priming
11. 'privacy_oncerns' for privacy concerns
12. 'self_esteem' for self-steem
13. 'self_report_bias' for self-report bias
14. 'network_effect' for network effect
15. 'truthful' for truthful

All categories must have the value of 1 if the category is observed in the response or 0 if it is not observed. Be sure that each tag is in simple quotes.

In that sense, the format should be:
{ 'uninformative': 0,

```
'SDB': 0,
'overest_others': 0,
'underest_own': 1,
'academic_integrity': 1,
'info_asymmetry': 0,
'AI_discussion_priming ': 0,
'privacy_concerns': 0,
'self_esteem': 1,
'self_report bias': 0,
'network_effect': 0,
'truthful': 0
}

// Constraints
If you classify a response as uninformative ('1'), then all other
categories must automatically be set to '0'.

If you are sure the response is informative ('0'), then you should
continue evaluating and assigning '1' or '0' to the other categories as
appropriate.
```

Appendix C Additional figures and tables

Table C.1. LLM validation metrics by configuration for Brandts and Cooper (2025).

Model	Temp.	Krippendorff's α	Accuracy	Precision	Recall	F1	N
Panel A. Zero-shot							
Claude	0	0.516	0.712	0.648	0.692	0.484	14
Claude	0.1	0.516	0.714	0.641	0.690	0.476	14
Claude	0.5	0.518	0.713	0.648	0.689	0.478	14
Claude	1	0.512	0.713	0.645	0.683	0.476	14
GPT	0	0.473	0.675	0.608	0.656	0.435	14
GPT	0.1	0.476	0.677	0.608	0.654	0.439	14
GPT	0.5	0.477	0.674	0.605	0.665	0.436	14
GPT	1	0.471	0.668	0.607	0.655	0.434	14
GPT	1.2	0.474	0.672	0.594	0.655	0.440	14
Panel B. Few-shot							
Claude	0	0.518	0.742	0.658	0.687	0.492	14
Claude	0.1	0.515	0.742	0.649	0.680	0.487	14
Claude	0.5	0.517	0.741	0.657	0.683	0.491	14
Claude	1	0.519	0.741	0.656	0.683	0.492	14
GPT	0	0.491	0.692	0.606	0.677	0.436	14
GPT	0.1	0.493	0.693	0.601	0.673	0.434	14
GPT	0.5	0.488	0.696	0.610	0.676	0.438	14
GPT	1	0.488	0.691	0.605	0.671	0.435	14
GPT	1.2	0.479	0.688	0.599	0.662	0.436	14

Notes: Entries report mean metrics across tags with at least 10 positive cases.N is the total categories evaluated. Precision and recall are macro averages over classes 0 and 1. Within each table, Panel A reports zero-shot prompting and Panel B reports few-shot prompting.

Table C.2. LLM validation metrics by configuration for Ederer and Schneider (2022).

Model	Temp.	Cohen's κ	Accuracy	Precision	Recall	F1	N
Panel A. Zero-shot							
Claude	0	0.757	0.920	0.863	0.914	0.878	3
	0.1	0.757	0.920	0.863	0.914	0.878	3
	0.5	0.753	0.918	0.862	0.912	0.876	3
	1	0.761	0.922	0.864	0.916	0.880	3
GPT	0	0.754	0.924	0.866	0.896	0.877	3
	0.1	0.758	0.926	0.869	0.898	0.879	3
	0.5	0.755	0.926	0.865	0.899	0.877	3
	1	0.786	0.932	0.881	0.914	0.893	3
	1.2	0.755	0.928	0.872	0.888	0.877	3
Gemini	0	0.648	0.884	0.844	0.874	0.819	3
	0.1	0.685	0.900	0.856	0.887	0.839	3
	0.5	0.676	0.900	0.855	0.873	0.835	3
	1	0.619	0.880	0.840	0.850	0.804	3
	1.2	0.642	0.884	0.845	0.868	0.816	3
Panel B. Few-shot							
Claude	0	0.707	0.898	0.841	0.895	0.852	3
	0.1	0.707	0.898	0.841	0.895	0.852	3
	0.5	0.708	0.898	0.843	0.895	0.852	3
	1	0.712	0.900	0.844	0.897	0.854	3
GPT	0	0.762	0.922	0.863	0.919	0.880	3
	0.1	0.745	0.918	0.859	0.902	0.872	3
	0.5	0.733	0.920	0.860	0.882	0.866	3
	1	0.723	0.914	0.850	0.887	0.861	3
	1.2	0.746	0.918	0.859	0.901	0.872	3
Gemini	0	0.658	0.884	0.854	0.853	0.825	3
	0.1	0.648	0.882	0.851	0.850	0.821	3
	0.5	0.668	0.886	0.856	0.867	0.831	3
	1	0.661	0.886	0.858	0.852	0.828	3
	1.2	0.659	0.886	0.853	0.856	0.826	3

Notes: Entries report mean metrics across tags with at least 10 positive cases. Precision and recall are macro averages over classes 0 and 1. N denotes the number of tags included in the average.

Table C.3. LLM validation metrics by configuration for Ling et al. (2025).

Model	Temp.	Cohen's κ	Accuracy	Precision	Recall	F1	N
Panel A. Zero-shot							
Claude	0	0.730	0.904	0.884	0.861	0.813	5
	0.1	0.719	0.900	0.878	0.855	0.809	5
	0.5	0.709	0.896	0.872	0.850	0.803	5
	1	0.710	0.896	0.875	0.851	0.804	5
GPT	0	0.398	0.781	0.712	0.698	0.651	5
	0.1	0.394	0.778	0.706	0.697	0.648	5
	0.5	0.409	0.781	0.711	0.706	0.657	5
	1	0.394	0.778	0.703	0.697	0.649	5
	1.2	0.392	0.780	0.707	0.695	0.654	5
Gemini	0	0.757	0.914	0.886	0.879	0.826	5
	0.1	0.743	0.904	0.878	0.884	0.818	5
	0.5	0.758	0.911	0.884	0.887	0.826	5
	1	0.784	0.921	0.896	0.901	0.837	5
	1.2	0.768	0.913	0.894	0.895	0.830	5
Panel B. Few-shot							
Claude	0	0.458	0.877	0.725	0.748	0.724	5
	0.1	0.697	0.896	0.844	0.863	0.847	5
	0.5	0.574	0.883	0.785	0.808	0.785	5
	1	0.717	0.896	0.868	0.856	0.805	5
GPT	0	0.702	0.885	0.859	0.850	0.803	5
	0.1	0.689	0.881	0.850	0.848	0.797	5
	0.5	0.702	0.885	0.859	0.854	0.804	5
	1	0.687	0.881	0.855	0.843	0.797	5
	1.2	0.740	0.900	0.879	0.870	0.832	5
Gemini	0	0.772	0.917	0.899	0.888	0.831	5
	0.1	0.743	0.912	0.900	0.870	0.814	5
	0.5	0.728	0.904	0.881	0.867	0.810	5
	1	0.748	0.919	0.905	0.870	0.816	5
	1.2	0.734	0.906	0.885	0.864	0.813	5

Notes: Entries report mean metrics across tags with at least 10 positive cases. Precision and recall are macro averages over classes 0 and 1. N denotes the number of tags included in the average.

Appendix D How to use Assign Categories

This section only describes how to assign categories using the developed framework, as the present paper focuses on evaluating LLM performance in classifying responses from the three empirical applications presented herein. Nonetheless, in other contexts, the *Create*

Categories option may be useful for initially defining the labels that will later be employed for classification. Crucially, this option follows a structured procedure that makes the category-creation process explicit, documented, and replicable: the researcher specifies the coding objective, provides illustrative examples of the target construct, and the framework derives candidate categories through a systematic inference process rather than relying solely on researcher intuition. This reduces the risk that the resulting categories are overly ad hoc or idiosyncratic, since any researcher following the same inputs and steps can reproduce the same label set. Additionally, because *Create Categories* follows a procedure similar to that of the *Assign Categories* option, the explanation that follows will also facilitate understanding of this functionality.

Figure D.1 shows how the tool will look after uploading the CSV file with the responses to be classified. It also shows the first observations as a way to help the user know if the CSV is the correct file. The *Reset* button can be used for reloading the page and deleting all the files created from previous uses. This is particularly useful if the researcher wants to analyze a completely different investigation or dataset.

Additionally, the tool provides two options: *Create categories* for generating and defining thematic categories from the uploaded responses, and *Assign categories* for automatically classifying each response into the previously created categories or the predefined categories established by the investigator.

If the Assign categories option is selected, the next step is to select the prompting strategy and the LLM model, and to enter the corresponding API key required to use the provider.

After that, the tool will show the prompt configuration section. As can be observed in Figure D.3, each section (Role, Context, Classification Task, Format, or Constraints) has its own module. Here, the user can either choose to write the section or upload a TXT file.

Format is the only section that has an additional option. For the format to be valid, it must be specified as a JSON structure; thus, to avoid issues related to this specification, we also implemented a user-friendly alternative. With this option, the researcher only needs to enter the categories and their corresponding classification, and the program will internally convert that list into a valid JSON format. Figure D.4 shows how this option appears.

When the prompt is complete, the code will process the data using each selected temperature. As described in the main manuscript, there are two ways in which the user can choose to process the data. The first one is *line by line*; Figure D.5 shows how each observation is processed. The second one is by batch. Figure D.6 illustrates how the tool will look if batches are submitted successfully.

In both options, the button *Reset and edit prompt* must be selected if the user wants

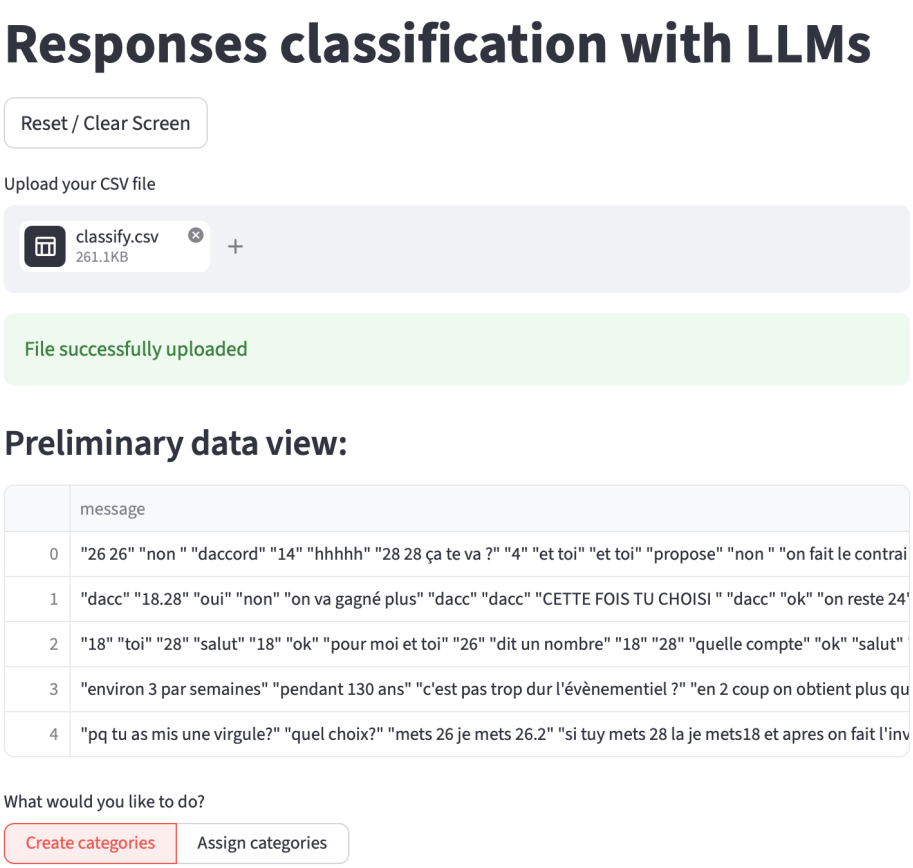

Figure D.1. Tool screen after uploading the csv with the responses that need classification

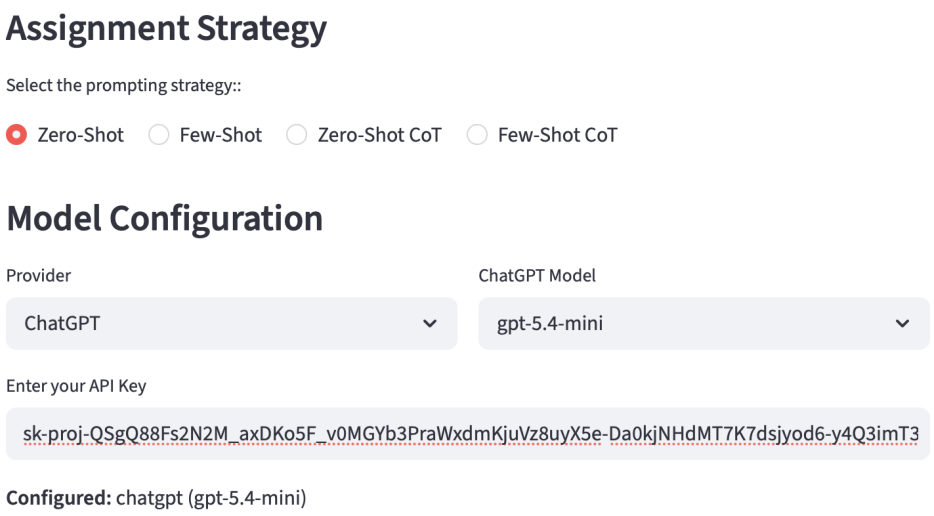

Figure D.2. Prompt type, model selection, and API key entry

Note: Cell annotation shows winning configuration (shot@temperature).

to modify the prompt or enter a new one. The user must download all the results before clicking it; otherwise, the option for downloading the new results will disappear.

A final section that will appear after selecting the LLM and typing the API key is the *Batches pending* component. Here, the user can observe the state of the batches, how many observations have been processed, and whether the results are ready for download. When the results are downloaded, that batch will be removed from the list.

Appendix E How to use *Create Categories*

Create categories has a procedure similar to that of *Assign categories*. After choosing the LLM and entering the API key, the only difference in the prompt configuration compared to the other option is that, instead of specifying the *Classification* task, the user must define the *Create categories* task. Figure E.1 shows this section and an example of how this task can be defined.

After completing the prompt and instructing the program to create the categories, the page will display the complete prompt, as shown in Figure E.2.

After this, the results corresponding to each selected temperature will appear. Unlike the *Assign categories* task, this option displays the created categories and their respective definitions in a table. The download option is still available; it will download the table in

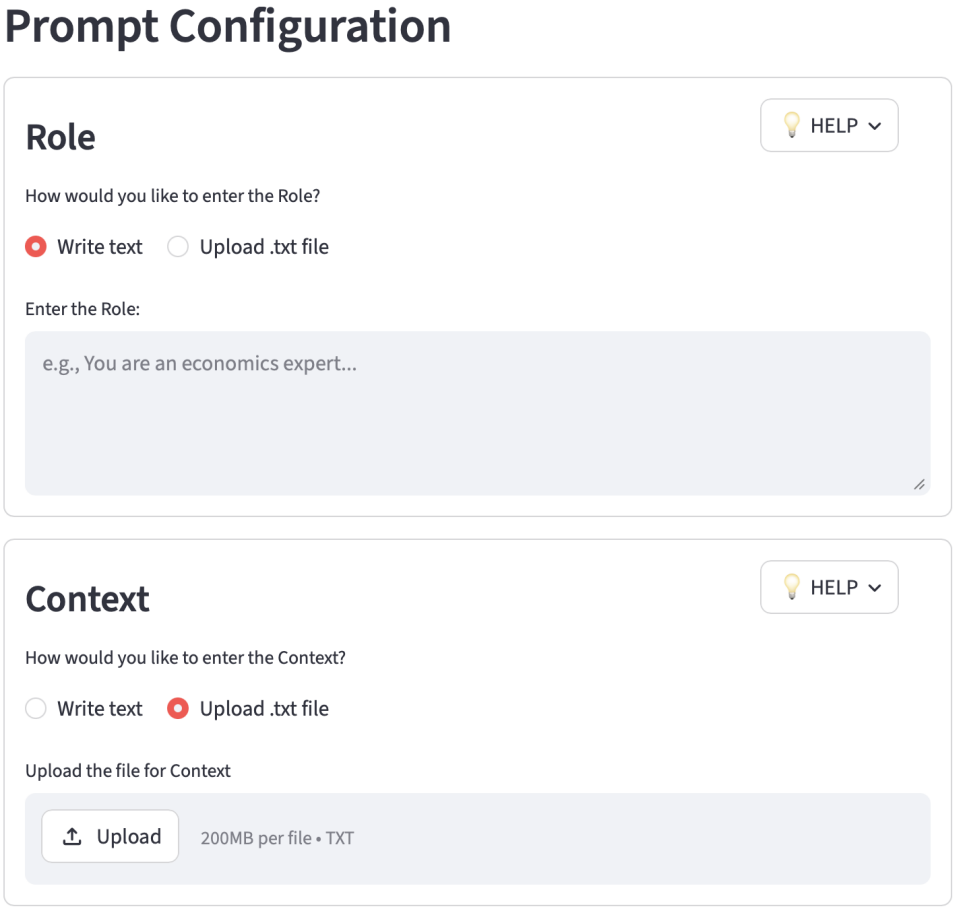

Figure D.3. Prompt configuration

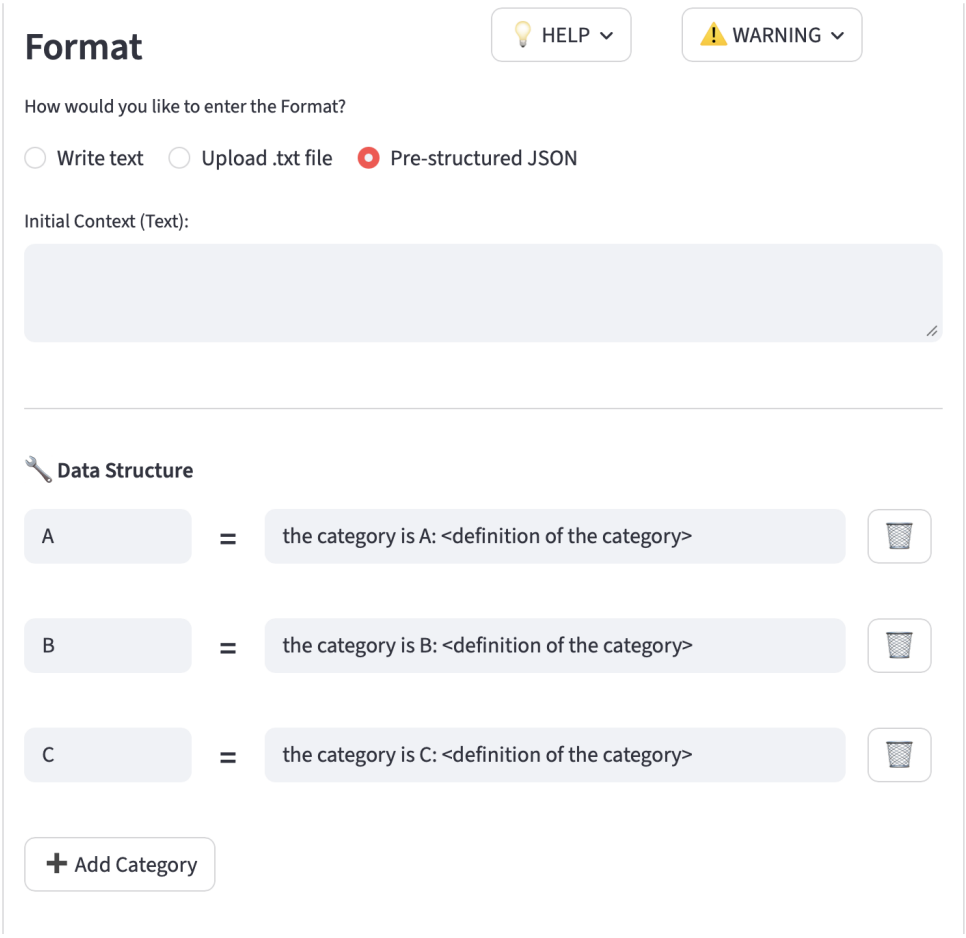

Figure D.4. Third option for FORMAT

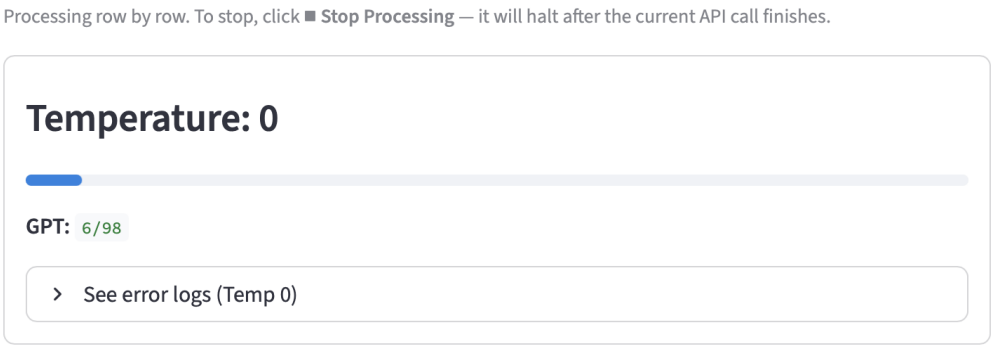

Figure D.5. *Processing row by row* option for temperature 0

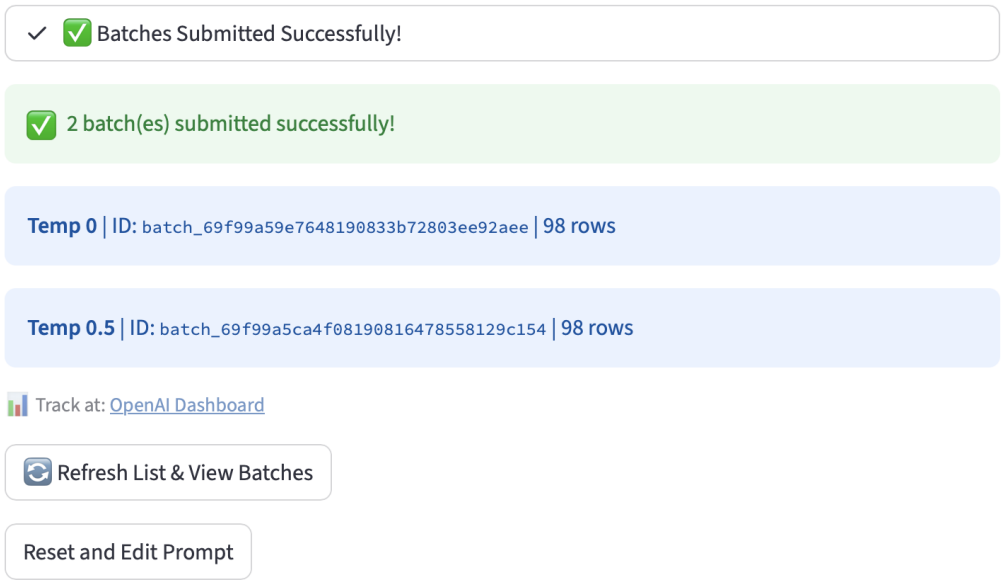

Figure D.6. Processing with batches option

Note: Cell annotation shows winning configuration (shot@temperature).

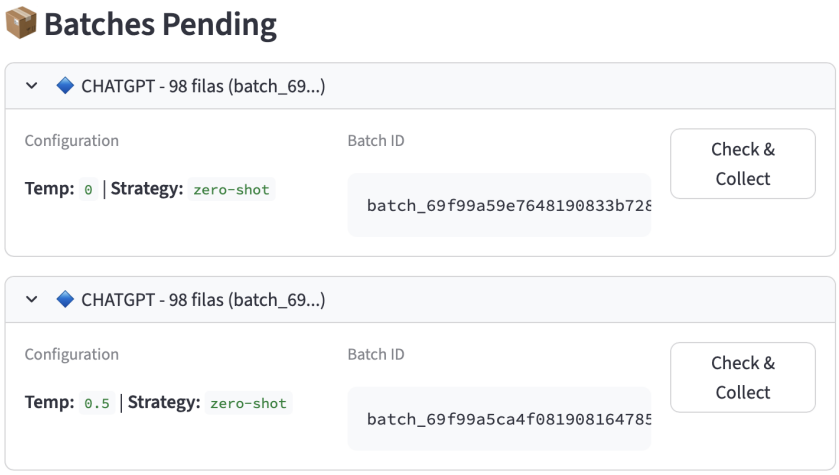

Figure D.7. Batches pending section

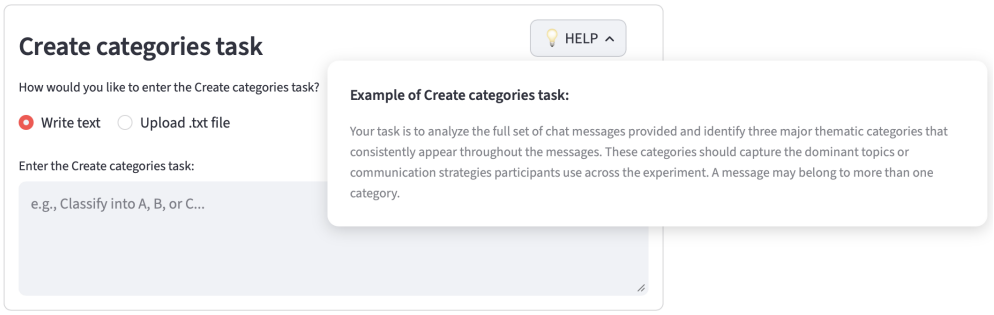

Figure E.1. Changed Prompt Section

Final Generated Prompt

```
You are a researcher with expertise in qualitative analysis. You also have experie
The present experiment aims to investigate how the strategic environment—whether i

In each period, participants simultaneously chose actions, with payoffs determined


Your task is to analyze the full set of chat messages provided and identify three
You must return the three categories you identify strictly as a Python dictionary,
{'A': 'the category is A: <definition of the category>', 'B': 'the category is B:
You must:

Analyze the complete content.
Identify the three most prevalent thematic categories.
Return them only in the required Python dictionary format.
After this first step (creating the categories), further instructions will follow
.
```

Figure E.2. Final prompt

CSV format. Figure E.3 shows an example of the final result.

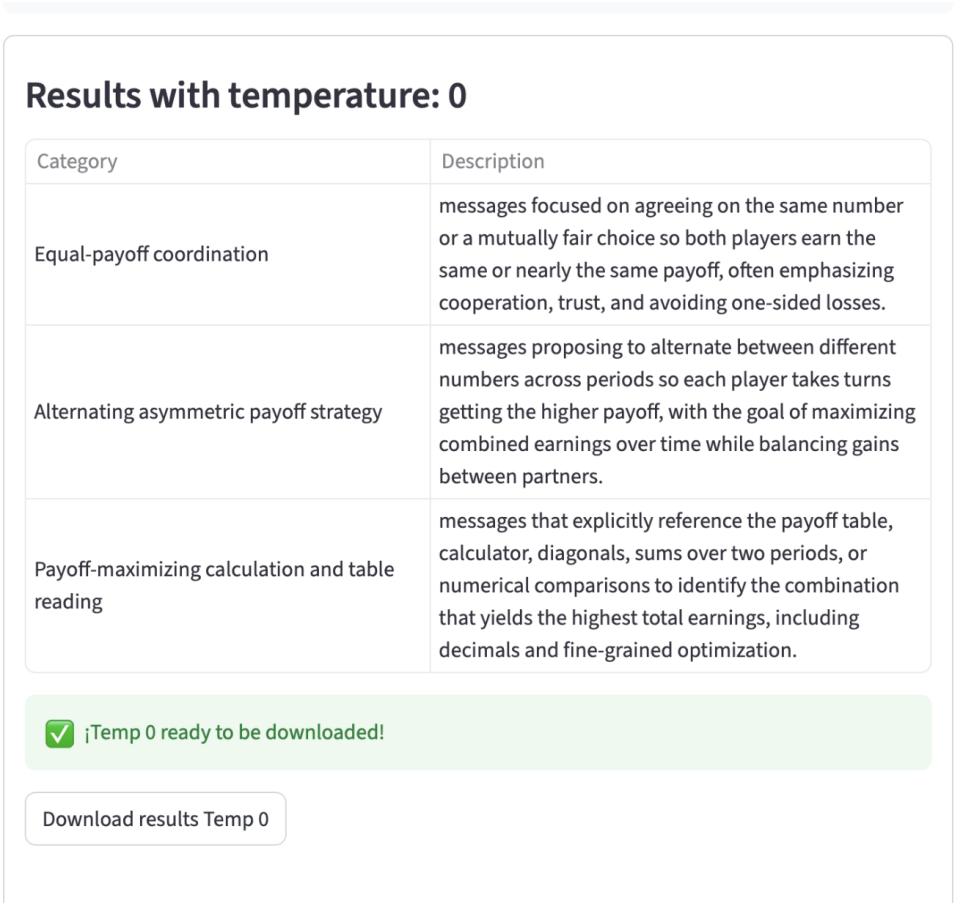

Figure E.3. Results

References

Brandts, J. and Cooper, D. J. (2025). Managerial leadership, truth-telling and efficient coordination. *The Economic Journal*, 135(670):1942–1979.

Ederer, F. and Schneider, F. (2022). Trust and promises over time. *American Economic Journal: Microeconomics*, 14(3):304–320.

Ling, Y., Kale, A., and Imas, A. (2025). Underreporting of AI use: The role of social desirability bias. *Available at SSRN 5464215*.